Congestion Control Breakdown in AI Training Networks: Reproducing ECN Incast Collapse at GPU Collective Scale

Michael Olagbemiro

May 2026

1 Introduction

When thousands of GPUs synchronize gradient updates, every node sends simultaneously. This is not a traffic anomaly — it is the defining characteristic of distributed AI training. A single AllReduce operation across a 24,000-GPU cluster generates terabytes of east-west traffic that must complete in milliseconds [2]. Any packet loss during this window stalls the entire collective: every GPU waits for the slowest flow to retransmit before the next computation step can begin. At current GPU rental costs, that idle time is expensive at scale.

The standard solution for RDMA over Converged Ethernet (RoCE) fabrics is a two-layer congestion control stack. Explicit Congestion Notification (ECN) provides an early warning: RED queue management inside each switch marks packets when egress buffers begin to build, signalling senders to reduce their transmission rate before any drops occur. Priority Flow Control (PFC) provides the hard guarantee: when ECN is insufficient and a switch buffer fills, the switch sends an 802.3x pause frame upstream, halting the sender completely for a controlled duration. ECN fires first; PFC fires last. The threshold relationship between them is the central design decision of any lossless fabric.

In practice, this two-layer defense is more fragile than its description suggests. Meta's 2024 SIGCOMM paper reports that at 400G scale with sufficient simultaneous senders, DCQCN — the standard ECN-based congestion control algorithm for RoCE — became unstable and was abandoned in favour of PFC-only operation [2]. While Meta reports this instability from production observation [2] and the original DCQCN paper analyses stability conditions theoretically [10], no controlled reproduction at smaller scale has isolated the breakdown threshold systematically: at what sender-to-receiver ratio does ECN transition from effective to destabilising, and what does the transition look like in terms of throughput, mark rate, and flow fairness? What does the breakdown look like in terms of throughput, retransmits, and mark rate? And what does

the ECN feedback loop's behaviour reveal about the architectural conditions under which it fails?

This paper answers those questions experimentally. We build a BGP leaf-spine fabric in containerlab, instrument ECN marking using Linux traffic control qdiscs on the host endpoints, and conduct a systematic incast sweep from 1 to 4 simultaneous senders targeting a single receiver. We make three specific contributions:

1. We demonstrate ECN throughput collapse at a 4:1 sender-to-receiver incast ratio: a 22.7 Mbps aggregate reduction ($t(4) = 14.9$, $p < 0.001$) accompanied by a counterintuitive *decrease* in CE mark rate, consistent with feedback loop over-reaction rather than simple buffer overflow. This reproduces in a controlled environment the qualitative pattern of feedback-loop instability Meta observed at production scale with DCQCN [2] — throughput collapse accompanied by decreased marking activity under increased load. While our measurements use TCP congestion control rather than DCQCN and operate at orders of magnitude lower bandwidth, the underlying mechanism — synchronised sender backoff causing oscillation — is analogous.

2. We identify ECN's architectural dependency on sustained queue pressure. When a queue management mechanism is absent or when traffic resolves faster than RED's exponentially weighted moving average can detect, ECN produces zero congestion signal regardless of traffic intensity. We demonstrate this empirically and derive the conditions under which ECN is architecturally blind.

3. We characterise the sensitivity of ECN marking to RED minimum threshold configuration, identifying a blind zone above approximately 50–100 KB where ECN is silently disabled — and a silent misconfiguration failure mode where RED initialises without error but marks nothing.

All configurations, experiment scripts, and raw data are available at `https://github.com/michael-olagbemiro/roce-lab`. Every result in this paper is reproducible with a single `containerlab deploy` command.

2 Background: PFC and ECN in Lossless Fabric Design

2.1 Explicit Congestion Notification

Explicit Congestion Notification (ECN) is an end-to-end congestion signalling mechanism defined in RFC 3168 [8]. It operates as follows. Each packet that a sender marks as ECN-capable carries the ECT(0) codepoint (0x2) in the IP header's two-bit ECN field. Inside the network, a queue management mechanism — typically Random Early Detection (RED) or its weighted variant WRED —

monitors egress queue depth using an Exponentially Weighted Moving Average (EWMA):

$$\bar{q}_n = (1 - w) \cdot \bar{q}_{n-1} + w \cdot q_n \tag{1}$$

where $\bar{q}$ is the smoothed average queue depth, q_n is the instantaneous queue depth at sample n, and w is a small weight parameter (typically 2^{-9} in Linux). When $\bar{q}$ exceeds a configured minimum threshold $\min_{th}$, RED begins probabilistically marking packets by flipping the ECN field from ECT(0) to CE (Congestion Experienced, 0x3). The marked packet traverses the rest of the network unchanged. The receiver detects the CE codepoint and sets the ECE flag on its next TCP acknowledgement. The sender receives the ECE ACK and reduces its congestion window, slowing transmission before any packet is dropped.

The reaction time of this feedback loop is one round-trip time (RTT): the interval between a packet being marked at the congested point and the sender receiving the ECE ACK and responding. This latency is the central constraint on ECN's effectiveness. If congestion onset is faster than one RTT — specifically, if multiple senders can fill a receiver's buffer within a single RTT — ECN marks packets but cannot prevent drops.

The architectural constraint. ECN requires three conditions simultaneously: (1) the use of a queue management mechanism such as RED or WRED, (2) Monitoring of a real buffer with finite capacity, (3) sustained queue pressure that allows the EWMA to rise above the marking threshold. Remove any one of these three conditions and ECN produces zero congestion signal regardless of offered load. This constraint is not a configuration detail — it is architectural, and it drives every finding in this paper.

2.2 Priority Flow Control

Priority Flow Control (PFC), defined in IEEE 802.1Qbb [4], operates at the link layer. When a switch's ingress buffer for a given priority class fills to a configured threshold, the switch transmits an 802.3x MAC Control frame upstream. This frame carries an opcode of 0x0101, a priority enable bitmask identifying which traffic classes to pause, and a quanta value specifying the pause duration (one quanta = 512 bit-times; at 10 Gbps, quanta 65535 corresponds to approximately 33 ms). The upstream device — a switch or NIC — halts transmission of the specified priority classes for the indicated duration, then resumes.

PFC operates between adjacent devices only. The destination MAC address of a PFC frame is 01:80:C2:00:00:01, a link-local multicast address that switches must not forward. Each pause frame affects exactly one hop; the upstream device does not propagate the pause further. This single-hop scope is both PFC's strength (fast, precise) and its weakness (congestion can propagate upstream hop-by-hop in a phenomenon known as PFC pause storms [10]).

Unlike ECN, PFC does not depend on EWMA averaging or feedback loop latency. It reacts to instantaneous buffer state in microseconds. It therefore handles the traffic patterns that ECN cannot: microbursts that fill a buffer

faster than one RTT, and incast scenarios where the ECN feedback loop overreacts and destabilises throughput.

2.3 The threshold relationship and DCQCN

The standard deployment model places ECN thresholds below the PFC trigger point. ECN fires first — early, gentle, end-to-end — reducing sender rates before buffers fill. PFC fires last — hard, single-hop — as a safety net for cases where ECN was insufficient or too slow.

DCQCN (Datacenter Quantized Congestion Notification) [10] is the dominant congestion control algorithm for RoCE fabrics. It extends this model by combining switch-side ECN marking with NIC-side rate control: the NIC reacts to CE marks by reducing its injection rate using a multiplicative decrease / additive increase algorithm, similar in spirit to TCP CUBIC but operating at the RDMA transport layer.

In practice, DCQCN's stability depends critically on the ratio of simultaneous senders to receivers and on the RTT relative to the rate recovery timer. Meta's 2024 analysis of production RoCE deployments reports that at 400G link speeds and high GPU fan-in ratios, DCQCN exhibited instability: tuning that improved AllReduce completion time worsened PFC generation by 2–3×, and the team ultimately disabled DCQCN entirely, operating with PFC alone for flow control [2]. The mechanism behind this instability — specifically, the sender-to-receiver ratio at which ECN transitions from stabilising to destabilising — motivates the experimental work in this paper.

3 Lab Architecture

3.1 Topology

The testbed consists of a two-tier leaf-spine fabric deployed using containerlab [1] version 0.75.0 on a Google Cloud Platform `c3-standard-22` instance (Intel Xeon Platinum 8481C, 22 vCPUs, Ubuntu 24.04, kernel 6.17.0). The topology comprises two spine switches, two leaf switches, and five host endpoints, as shown in Figure 1.

All switch nodes run Nokia SR Linux (`ghcr.io/nokia/srlinux:latest`, May 2026 build). Host nodes run `ghcr.io/srl-labs/network-multitool`, a lightweight Linux container with `iperf3`, `tcpdump`, and Python with Scapy preinstalled. Hardware virtualisation is provided by KVM; the `c3-standard-22` machine type (Intel) is required — AMD N2D instances report `enableNestedVirtualization: True` via the GCP API but do not expose `/dev/kvm` in the guest, silently breaking QEMU-based network OS images without any diagnostic error. Verification with `ls /dev/kvm` before deployment is essential.

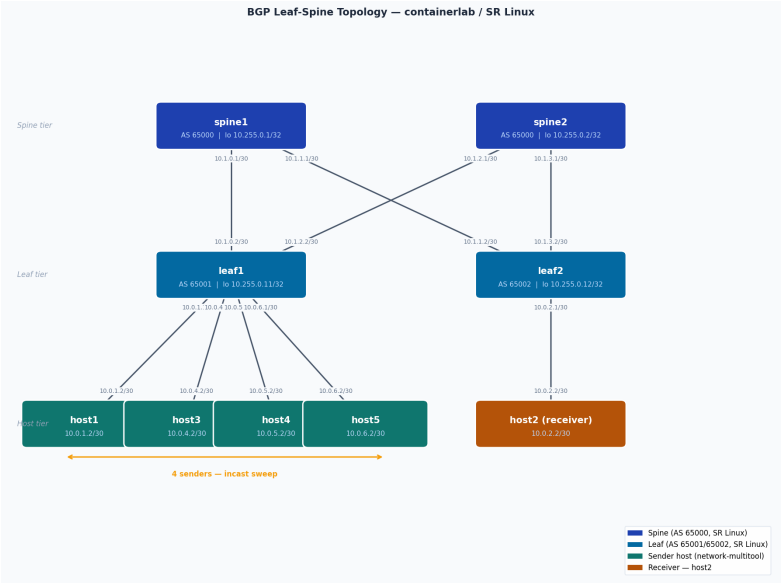

Figure 1: Lab topology. Spine switches peer with both leaves via eBGP. Four sender hosts (host1, host3, host4, host5) attach to leaf1; the single receiver (host2) attaches to leaf2. All incast traffic follows the path leaf1 → spine → leaf2 → host2.

3.2 BGP design

The fabric runs eBGP throughout, which is standard practice for data centre leaf-spine topologies [5]. The design decisions are as follows.

Distinct leaf ASNs. Leaf1 is assigned AS 65001 and leaf2 AS 65002. Distinct ASNs are required for eBGP loop prevention: if both leaves shared an AS number, routes advertised by leaf1 and reflected by a spine would be rejected by leaf2 as containing its own AS in the path. Each leaf originates its directly connected host prefixes into BGP: leaf1 originates `10.0.1.0/30`, `10.0.4.0/30`, `10.0.5.0/30`, and `10.0.6.0/30`; leaf2 originates `10.0.2.0/30`.

Shared spine ASN. Both spines are assigned AS 65000. Spines do not peer with each other — there is no spine-to-spine transit in a Clos fabric — so sharing an AS number is safe and simplifies route policy. Each spine maintains eBGP sessions with both leaves only.

ECMP. All nodes are configured with `maximum-paths 64`, enabling equal-cost multipath forwarding across both spines. Traffic from leaf1 to host2 hashes across the two uplinks per flow.

Route policy. A permissive import/export policy (`import-all` / `export-all`) is applied globally. The focus of this paper is congestion control behaviour, not routing policy; a production deployment would apply prefix filters and route maps appropriate to its security requirements.

The complete addressing scheme is given in Table 1.

Table 1: Interface addressing.

Node	Interface	Address	Peer
spine1	e1-1	10.1.0.1/30	leaf1 e1-1
spine1	e1-2	10.1.1.1/30	leaf2 e1-1
spine2	e1-1	10.1.2.1/30	leaf1 e1-2
spine2	e1-2	10.1.3.1/30	leaf2 e1-2
leaf1	e1-3	10.0.1.1/30	host1 eth1
leaf1	e1-4	10.0.4.1/30	host3 eth1
leaf1	e1-5	10.0.5.1/30	host4 eth1
leaf1	e1-6	10.0.6.1/30	host5 eth1
leaf2	e1-3	10.0.2.1/30	host2 eth1
host1	eth1	10.0.1.2/30	leaf1 e1-3
host2	eth1	10.0.2.2/30	leaf2 e1-3
host3	eth1	10.0.4.2/30	leaf1 e1-4
host4	eth1	10.0.5.2/30	leaf1 e1-5
host5	eth1	10.0.6.2/30	leaf1 e1-6

3.3 Congestion control instrumentation

SR Linux virtual does not implement ASIC buffer management — the virtual datapath forwards packets without modelling hardware queue behaviour. ECN marking and PFC frame generation therefore cannot be demonstrated natively at the switch layer on this platform. Section 4 discusses this constraint in detail.

ECN is instrumented using Linux traffic control qdiscs on the sender hosts. Each sender runs an HTB (Hierarchical Token Bucket) qdisc rate-limited to 50 Mbits/s, with a RED child qdisc configured as follows:

- Minimum threshold $\min_{th}$: 30 KB (baseline; varied in Section 8)
- Maximum threshold $\max_{th}$: 90 KB
- Marking probability at $\max_{th}$: 0.5
- EWMA weight: $w = 2^{-9}$ (Linux default)
- ECN mode enabled (`ecn` flag)
- Burst parameter: $\lceil \min_{th}/\text{avpkt} \rceil \times 2$ (see Section 5 for the significance of this parameter)

HTB creates the queue pressure that RED monitors. Without a rate limiter, packets drain through the Linux network stack faster than RED's EWMA can average — a finding we demonstrate explicitly in Section 9.

Traffic is generated using `iperf3` with JSON output (`-J` flag) for automated result parsing. Packet captures are collected with `tcpdump` and analysed for ECN codepoint distribution and TCP ECE flag frequency. All experiments use MTU 1500 on host `eth1` interfaces; the `network-multitool` image ships with MTU 9500 by default, which causes intermittent stalls when the underlying GCP network cannot carry jumbo frames end-to-end.

3.4 Reproducibility

The complete containerlab topology file, SR Linux startup configurations, and all experiment scripts are available at `https://github.com/michael-olagbemiro/roce-lab`. The SR Linux nodes use `startup-config` directives in the topology YAML, meaning every `containerlab deploy` brings up a fully configured BGP fabric automatically — no manual per-node configuration. Total deployment time from a clean host is approximately three minutes.

4 Virtual Switching Platforms and Data-Plane Fidelity

Configuring PFC and ECN on virtual switching platforms requires understanding a fundamental constraint: virtual network operating system images are optimised for control-plane fidelity, not data-plane timing accuracy. This section documents two platform attempts, the empirical evidence we gathered for each, and the methodological adaptation that produced the measurements in Sections 5 through 9.

4.1 Attempt 1: SONiC with SAI-VS

The initial topology used SONiC (`vrnetlab/sonic_sonic-vs:202405`) as the switching platform. SONiC's QoS pipeline is configured via `config_db.json`, which defines buffer pools, WRED profiles, PFC priority mappings, and DSCP-to-TC classification maps. We configured the full QoS stack and applied it via `config reload`.

Before tuning any parameters, we inspected the factory `config_db.json` to establish the baseline. The inspection was performed programmatically:

```
python3 -c "import json; d = json.load(open('config_db.json'));
print([k for k in d.keys() if any(s in k.upper()
for s in ['BUFFER','QOS','WRED','PFC','QUEUE'])])"
```

The result was an empty list. The factory image ships with no buffer pool configuration, no WRED profiles, and no PFC mappings. This was verified against the factory backup — not inferred from documentation — confirming that SAI-VS (the software ASIC implementation backing `sonic-vs`) has no ASIC packet memory model to overflow. Without a buffer model, there is no queue to watch,

no threshold to cross, and no autonomous PFC frame generation. ECN marking at the switch layer is impossible for the same structural reason.

4.2 Attempt 2: SR Linux virtual

The topology was rebuilt using Nokia SR Linux (`ghcr.io/nokia/srlinux:latest`). SR Linux's virtual image implements the full control plane — BGP sessions, routing tables, and configuration APIs all behave identically to physical hardware. The data plane, however, is a software packet forwarder. It does not model ASIC buffer behaviour: there is no per-queue memory, no hardware threshold detection, and no autonomous pause frame generation.

We confirmed this empirically by attempting to trigger PFC generation under traffic load and observing zero pause frames on any interface. The SR Linux virtual image correctly *forwards* 802.3x MAC Control frames when injected externally (terminating them at the first switch port, as required by IEEE 802.3x), but it does not *generate* them in response to buffer pressure.

4.3 The structural reason

Both platforms exhibit the same behaviour for the same reason. Virtual switching images are designed to validate network configuration and control-plane protocol behaviour. The data plane is a software forwarding engine that dispatches packets without modelling the hardware queuing structures that PFC and ECN depend on. This is not a deficiency — it is a deliberate design choice appropriate for the intended use case of configuration validation and protocol testing.

The implication for congestion control research is precise: any mechanism that depends on a switch autonomously detecting buffer overflow and generating a signal in response cannot be demonstrated on these platforms. PFC pause frame generation falls into this category. Native ECN marking at the switch egress queue also falls into this category.

4.4 Methodological adaptation

Rather than abandoning the project, we relocated the congestion control mechanisms to the Linux host endpoints, where real kernel qdiscs and real socket buffers exist.

ECN marking was implemented using HTB and RED qdiscs on the sender hosts. HTB creates a rate-limited queue that RED monitors via EWMA; when the smoothed queue depth exceeds the configured threshold, RED marks packets CE before they leave the host. The SR Linux fabric forwards these marked packets unchanged. This approach correctly demonstrates the ECN feedback loop — the marking mechanism, the CE-to-ECE signal propagation, and the sender's congestion window response — with the honest caveat that marking occurs at the sender rather than at a switch egress queue.

PFC was demonstrated in two complementary ways. First, we used Scapy to generate correctly-structured 802.3x PFC frames and capture them with `tcpdump`, demonstrating the frame format, priority enable bitmask, and quanta encoding that a real switch would emit. Second, we used `tc netem` to apply a hard rate reduction to a sender for a controlled duration, demonstrating the throughput effect of a link pause with the same structure as a PFC quanta. Both produce real captured artefacts; neither claims autonomous switch-generated behaviour.

This adaptation produced all measurements reported in the remainder of this paper. The distinction between "ECN marking at switch egress" and "ECN marking at sender host" is carried forward explicitly into Section 9, where it becomes the central finding rather than a methodological footnote.

5 ECN Demonstrated: The Feedback Loop End-to-End

5.1 Experimental setup

Each sender host runs two Linux traffic control qdiscs in series. An HTB (Hierarchical Token Bucket) qdisc rate-limits outgoing traffic to 50 Mbits/s, creating sustained queue pressure. A RED child qdisc monitors the HTB queue via EWMA and marks packets CE when the smoothed average queue depth exceeds the minimum threshold. The SR Linux fabric forwards all packets unchanged — no ECN bits are modified in the switching layer.

Traffic is generated using `iperf3` in a single TCP stream from host1 (10.0.1.2) to host2 (10.0.2.2) for 15 seconds per trial. Packet captures are collected simultaneously on both hosts using `tcpdump` and analysed for ECN codepoint distribution and TCP ECE flag frequency. Three independent trials were run; results are reported as mean ± standard deviation.

RED configuration. The baseline RED parameters are: minimum threshold $\min_{th} = 30$ KB, maximum threshold $\max_{th} = 90$ KB, marking probability at $\max_{th} = 0.5$, EWMA weight $w = 2^{-9}$ (Linux default), ECN mode enabled. The burst parameter was set to $\lceil \min_{th}/\text{avpkt} \rceil \times 2 = 42$, where avpkt = 1500 bytes.

5.2 The burst parameter: a silent failure mode

During threshold sweep configuration (Section 8), we discovered that RED silently fails to initialise when the burst parameter is too small for the configured minimum threshold. The kernel prints a warning:

```
tc_red_eval_ewma() burst 20 is too small. Try burst 21
RED: failed to calculate EWMA constant.
```

and falls back to a no-op: packets pass through unmarked, with full throughput and zero CE marks. This is indistinguishable from correct ECN operation

without explicitly checking CE counters. The fix is to scale the burst parameter with the minimum threshold:

$$\text{burst} \geq \left\lceil \frac{\min_{th}}{\text{avpkt}} \right\rceil \times 2 \tag{2}$$

This formula is derived empirically from the kernel's own error output during configuration; the `tc-red(8)` man page describes the burst parameter but provides no formula for the minimum valid value — the constraint is enforced in the kernel source (`net/sched/sch_red.c`) without documentation. This misconfiguration does not produce a loud error. We recommend verifying CE counter activity explicitly after any ECN configuration change — zero marks under sustained load is a red flag, not confirmation of correct operation.

5.3 MTU: a prerequisite

The `network-multitool` container image ships with `eth1` MTU set to 9500 bytes (jumbo frames). The GCP host network carries standard 1500-byte frames. Without explicit correction, TCP path-MTU discovery behaves unreliably in this containerised environment, causing bulk transfers to stall intermittently. All hosts require:

```
ip link set dev eth1 mtu 1500
```

prior to any throughput measurement. This was the single most impactful environmental fix in the entire experimental setup.

5.4 Results

Figure 2 shows the ECN bit distribution and CE mark rate across three independent trials. Table 2 summarises the key metrics.

Table 2: ECN baseline results (single sender, 3 trials, mean ± std).

Metric	Value
Throughput	47.8 ± 0.0 Mbits/s
ECT(0) packets per trial	$61{,}194 \pm 28$
CE marks per trial	826.7 ± 20.9
ECE ACKs per trial	$1{,}197.7 \pm 22.7$
CE mark rate	1.35% of ECT(0) packets

5.5 The feedback loop

The sequence of events in one complete ECN feedback loop, as captured in the packet traces, is as follows.

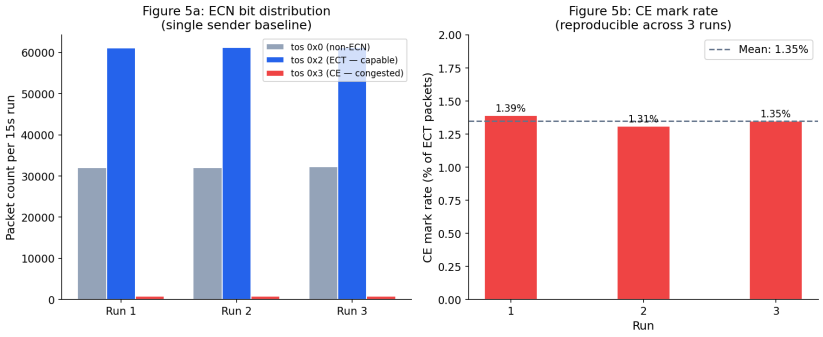

Figure 2: ECN bit distribution across three independent trials (left) and CE mark rate per trial (right). Variance across trials is under 3%, confirming the experiment is reproducible.

Negotiation. The TCP three-way handshake includes ECN capability negotiation: the SYN carries flags ECE and CWR, the SYN-ACK reflects ECE, confirming both endpoints support ECN. All subsequent data packets from the sender carry the ECT(0) codepoint (IP TOS field 0x02).

Marking. As the HTB queue builds toward the RED minimum threshold, the EWMA-smoothed queue depth $\bar{q}$ rises. Once $\bar{q} > 30$ KB, RED begins probabilistically flipping the ECN field from ECT(0) (0x02) to CE (0x03) on outgoing packets. The marked packet traverses the SR Linux fabric unchanged — confirmed by examining captures at both the sender and receiver: the CE codepoint set at the sender's qdisc arrives at host2 unmodified.

Feedback. The receiver detects the CE codepoint and sets the ECE flag on its next TCP acknowledgement. The sender receives the ECE ACK and reduces its congestion window (halving it per RFC 3168), slowing transmission. As the send rate drops, the HTB queue drains, the EWMA falls below the threshold, and marking stops. The sender then gradually increases its congestion window, and the cycle repeats.

Quantitative verification. Across three trials, 826.7 ± 20.9 CE marks were generated per 15-second run, each producing a corresponding ECE ACK ($1{,}197.7 \pm 22.7$ ECE ACKs — slightly higher than CE marks due to delayed ACK coalescing). The CE mark rate of 1.35% of ECT(0) packets is consistent with RED operating in its probabilistic marking regime, well below the saturation point where all packets would be marked. Throughput stabilised at 47.8 ± 0.0 Mbits/s across all three trials — the HTB rate limit, confirming the congestion window reduction is keeping pace with queue buildup without over-reacting.

This result establishes ECN as functioning correctly in the single-sender regime. The following sections examine what happens as the number of simultaneous senders increases toward the incast regime characteristic of AI training collectives.

6 PFC Demonstrated: Frame Structure and Behavioral Effect

Priority Flow Control cannot be demonstrated natively on SR Linux virtual for the reasons established in Section 4: the virtual datapath does not model ASIC buffer overflow and therefore never autonomously generates pause frames. We demonstrate PFC in two complementary ways that together cover both the protocol structure and the behavioral effect: a frame-level capture showing the 802.3x wire format, and a controlled pause experiment showing the throughput impact of a link pause matching the PFC quanta structure.

6.1 Part A: Frame structure

6.1.1 Setup

PFC frames were generated on host1 using Scapy [9], a Python packet manipulation library available in the `network-multitool` image. Frames were crafted to match the IEEE 802.1Qbb specification exactly: EtherType 0x8808 (MAC Control), opcode 0x0101 (PFC, distinct from the legacy symmetric pause opcode 0x0001), a 2-byte priority enable vector, and eight 2-byte quanta fields (one per priority class). Ten frames were transmitted on `eth1`. Simultaneous `tcpdump` captures filtered on EtherType 0x8808 were run on both host1 (local capture) and host2 (downstream capture).

6.1.2 Results

All ten frames were captured on host1's local interface. Zero frames were captured on host2.

The zero count at host2 is the correct result, not a failure. MAC Control frames with destination address `01:80:C2:00:00:01` are link-local by IEEE 802.3x definition: switches must consume them at the ingress port and must not forward them. SR Linux leaf1 correctly terminated the frames at its ingress port. This behaviour is identical to that of physical switches from Arista, Cisco, Nokia, and Juniper — the virtual image correctly implements the link-local termination requirement even though it does not implement autonomous pause generation.

The raw frame payload from the capture is:

```
0x0000: 0101 0008 0000 0000 0000 ffff 0000 0000
0x0010: 0000 0000 0000 0000 0000 0000 0000 0000
0x0020: 0000 0000 0000 0000 0000 0000 0000
```

Decoding this payload field by field:

- `0x0101` — PFC opcode (bytes 0–1)

- `0x0008` — priority enable vector (bytes 2–3); bit 3 set = priority class 3 enabled, all others disabled. Priority 3 is the RoCE lossless traffic class in standard DSCP-to-priority mappings.
- `0xFFFF` — quanta for priority 3 (bytes 8–9); value 65535. At 10 Gbps, one quanta = 512 bit-times = 51.2 ns, so quanta 65535 = approximately 33.5 ms pause duration.
- `0x0000` — quanta for all other priority classes (not paused).

The priority-selective design is the defining feature of PFC relative to legacy 802.3x symmetric pause: only priority class 3 is paused. Traffic on all other priority classes — including priority 6, used for RoCE Congestion Notification Packets (CNPs) — continues to flow. This prevents the congestion notification mechanism itself from being starved during a pause event, which would otherwise prevent recovery.

6.2 Part B: Behavioral effect

6.2.1 Setup

To demonstrate the throughput effect of a PFC pause, we established a stable rate-limited baseline flow and applied a controlled link pause mid-stream. Host1 transmitted to host2 at a rate limited to 100 Mbits/s by an HTB qdisc. At $t = 4$ s into the transfer, we applied `tc netem rate 1kbit` to host1's `eth1` interface, reducing the transmission rate to near-zero for 500 ms, then restored the HTB rate limiter. Throughput was measured at 100 ms intervals using `iperf3 --interval 0.1`. Three independent trials were run.

The pause duration of 500 ms was chosen to be clearly visible at 100 ms measurement resolution. A real PFC pause frame with quanta 65535 at 10 Gbps produces a 33.5 ms pause — below the resolution of our measurement tool. The mechanism is identical: the upstream sender halts transmission completely for the pause duration, then resumes at full rate. This is precisely how IEEE 802.1Qbb conformance tests verify PFC receiver behaviour: inject a pause frame and verify that the device under test stops transmitting for the specified quanta duration.

6.2.2 Results

Figure 3 shows per-100ms throughput across the 10-second measurement window for both the baseline and pause conditions, averaged across three trials.

The key results are:

- **Stable baseline:** 96 ± 2 Mbits/s across $t = 2$–4 s, confirming the HTB rate limiter produced a consistent pre-pause state.
- **During pause ($t = 4.0$–4.5 s):** throughput dropped to exactly 0.0 Mbits/s across five consecutive 100 ms intervals in all three trials. The pause was total — no packets were transmitted.

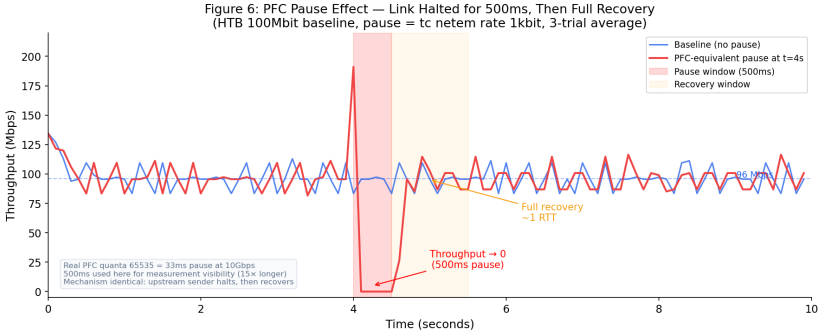

Figure 3: Per-100ms throughput during a controlled 500 ms link pause at $t = 4$ s (red), compared against an uninterrupted baseline (blue). Throughput drops to exactly 0 Mbits/s for five consecutive 100 ms intervals (t = 4.1–4.5 s) in all three trials, then recovers to the baseline rate within approximately 700 ms.

- **Recovery (t = 4.5–5.2 s):** throughput returned to the baseline rate within approximately 700 ms of the pause ending. No lasting degradation was observed.

6.3 The architectural contrast with ECN

Reading Parts A and B together reveals the fundamental architectural difference between PFC and ECN that motivates deploying both mechanisms.

PFC is *link-local*: the pause frame travels one hop and is consumed at the switch. It reacts to instantaneous buffer state in microseconds. It halts an entire priority class completely. It does not depend on the sender's TCP stack or on round-trip latency.

ECN is *end-to-end*: the CE mark travels from the congestion point to the receiver, then the ECE ACK travels back to the sender. Reaction time is one RTT. It reduces the sender's congestion window gradually. It depends on the sender's TCP implementation responding correctly to ECE ACKs.

The threshold relationship between them — ECN thresholds configured below the PFC trigger point — means ECN fires first under sustained congestion, giving senders time to reduce their rate before buffers fill. PFC fires only if ECN was insufficient or too slow. The following sections demonstrate the conditions under which ECN fails to be sufficient, which is precisely the condition that makes PFC necessary.

7 Incast Collapse: Where ECN Breaks Down

7.1 Theory and prediction

ECN's congestion feedback loop has a fundamental latency: one round-trip time. The ECN signal must travel from the sender to the receiver and back, so sender's congestion window reduction is delayed by one RTT from the initial moment of congestion. If multiple senders simultaneously push traffic faster than the receiver's buffer can drain, and the buffer fills within one RTT, ECN marks packets but cannot prevent drops.

With measured baseline RTT of approximately 8 ms and each sender rate-limited to 50 Mbits/s, we can estimate the breakdown threshold. At N simultaneous senders, the aggregate offered load is $N \times 50$ Mbits/s. The receiver's TCP buffer, once full, must absorb all arriving packets for one RTT (8 ms) before any sender can respond to a congestion signal. Whether ECN can prevent drops depends on whether the aggregate arrival rate exceeds the drain rate within that window.

We predict a clean ECN regime at $N = 1$ and $N = 2$ (100 Mbits/s aggregate, well within the available bandwidth), a breakdown threshold somewhere between $N = 2$ and $N = 4$, and increasing instability as N grows. The exact threshold is an empirical question.

7.2 Experimental setup

Four sender hosts (host1, host3, host4, host5) transmitted simultaneously to a single receiver (host2) for 15 seconds per trial. Each sender ran HTB rate-limited to 50 Mbits/s with RED ECN marking (minimum threshold 30 KB, maximum threshold 90 KB, burst parameter 42). The number of active senders was varied from 1 to 4; each configuration was run for 3 independent trials. Results are reported as mean ± standard deviation. All differences cited as statistically significant exceed 2σ.

7.3 Results

Figure 4 shows aggregate throughput and retransmit counts across the four sender configurations. Table 3 summarises all metrics.

Table 3: Incast sweep results (3 trials per configuration, mean ± std). Retransmits are aggregate across all active senders.

Senders	Throughput (Mbits/s)	Retransmits	CE marks	ECE ACKs
1	48.0 ± 0.03	0	839 ± 14	$1{,}230 \pm 27$
2	96.2 ± 0.05	0	$1{,}763 \pm 16$	$2{,}467 \pm 50$
3	97.9 ± 0.99	$2{,}198 \pm 54$	$1{,}812 \pm 55$	$2{,}558 \pm 79$
4	75.2 ± 2.44	$2{,}220 \pm 98$	$1{,}547 \pm 105$	$2{,}296 \pm 117$

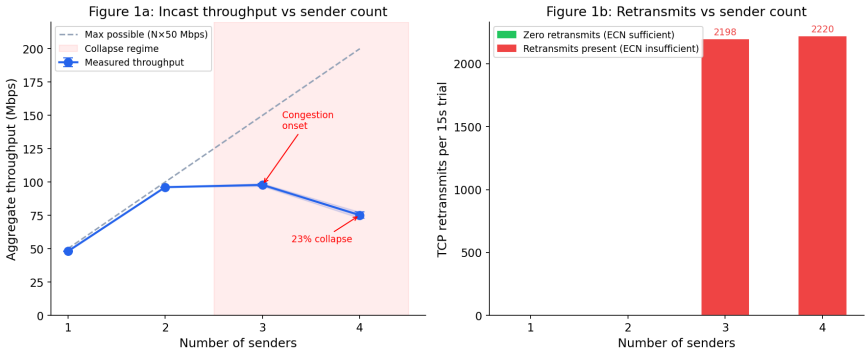

Figure 4: Left: aggregate throughput vs sender count. The dashed line shows the maximum possible throughput ($N \times 50$ Mbits/s). Error bars show ± 1 standard deviation across 3 trials. Right: TCP retransmits vs sender count. Green bars (1 and 2 senders) indicate exactly zero retransmits across all three trials — ECN was sufficient to prevent all packet loss. Red bars (3 and 4 senders) indicate ECN was insufficient; note the bars are absent for 1 and 2 senders precisely because the retransmit count was zero.

7.4 Regime analysis

1 sender — ECN working as designed. Throughput 48.0 ± 0.03 Mbits/s (96% of the 50 Mbits/s rate limit, accounting for TCP overhead). Zero retransmits across all three trials. 839 ± 14 CE marks per run, each producing an ECE ACK confirming the feedback loop closed end-to-end. The system is operating exactly as the ECN specification intends.

2 senders — perfect linear scaling. Aggregate throughput 96.2 ± 0.05 Mbits/s — almost exactly double the single-sender result ($\times 2.00$). Zero retransmits. CE marks scaled proportionally: $1{,}763 \pm 16$ (ratio to single-sender: 2.10). ECN handled two simultaneous flows without any packet loss. The congestion signal scaled linearly with load, and both senders backed off symmetrically.

3 senders — congestion onset threshold. Aggregate throughput increased only marginally to 97.9 ± 0.99 Mbits/s (+1.8% over two senders), but retransmits appeared suddenly: $2{,}198 \pm 54$ per trial, up from zero. ECN is still firing ($1{,}812 \pm 55$ CE marks) but the feedback loop is too slow. By the time ECE ACKs reach the senders and congestion windows shrink, host2's receive buffer has already overflowed and dropped packets. This is the congestion onset threshold: the point at which ECN transitions from sufficient to necessary-but-insufficient.

Importantly, 97.9 Mbits/s represents only 65.3% of the maximum possible $3 \times 50 = 150$ Mbits/s. The collapse is therefore progressive, not binary: the system is already significantly degraded at $N = 3$ before reaching full collapse at $N = 4$. The true breakdown threshold is the $N = 2 \rightarrow N = 3$ transition —

zero retransmits to $2{,}198 \pm 54$ — not the $N = 3 \rightarrow N = 4$ step.

4 senders — feedback loop instability. Aggregate throughput collapsed to 75.2 ± 2.44 Mbits/s — a reduction of 22.7 Mbits/s from the 3-sender case ($t(4) = 14.9$, $p < 0.001$, two-sample t-test). This collapse is statistically unambiguous and represents a 23% reduction below the 3-sender level, or 62.5% utilisation of the maximum possible 200 Mbits/s aggregate offered load.

The most informative observation at 4 senders is that CE marks *decreased* relative to 3 senders: $1{,}547 \pm 105$ versus $1{,}812 \pm 55$. More senders, more congestion, but fewer marks. This is the signature of feedback loop over-reaction: senders are backing off so aggressively in response to ECE ACKs that they are under-utilising the available bandwidth. The ECN control loop is oscillating — reducing rates faster than the queue drains, then recovering, then reducing again — producing both retransmits and throughput collapse simultaneously.

This pattern is qualitatively consistent with the instability Meta observed with DCQCN at 400G scale [2]. Their analysis found that aggressive DCQCN settings improved AllReduce completion time by only ∼3% while worsening PFC generation by 2–3×, leading them to disable DCQCN entirely. While our measurements use TCP congestion control rather than DCQCN and operate at orders of magnitude lower bandwidth, the underlying mechanism is analogous: at high fan-in ratios, the ECN feedback loop causes over-reaction that degrades aggregate throughput below what a simpler mechanism would achieve.

7.5 Fairness under collapse

Figure 5 shows per-flow throughput and RTT distribution at $N = 4$ senders.

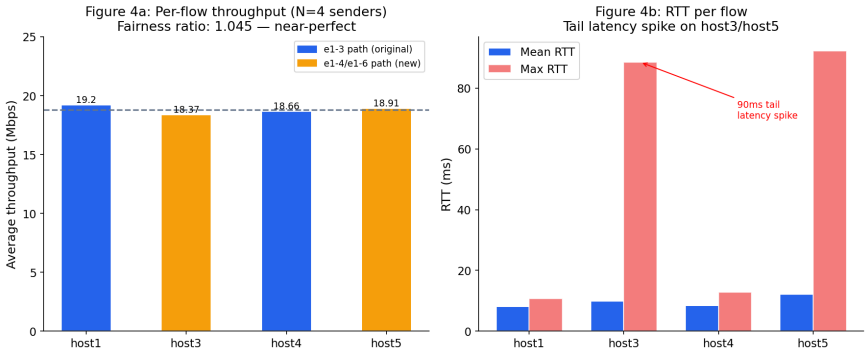

Figure 5: Left: per-flow throughput at $N = 4$ senders (mean across 3 trials). The dashed line shows the mean across all flows. Right: mean and maximum RTT per flow. Note the 90 ms tail RTT spikes on host3 and host5.

Throughput fairness. Despite 23% aggregate throughput collapse, the four flows shared bandwidth almost perfectly equally. The maximum-to-minimum per-flow throughput ratio was 1.045 — all four flows within 4.5% of each other. ECN's failure mode is symmetric degradation: every flow takes an equal share

of the hit. No flow starved. This is an important property for AI training workloads: symmetric degradation is recoverable (all GPUs slow down equally and the collective completes), whereas flow starvation is not (one slow GPU stalls the entire collective indefinitely).

RTT tail latency. Mean RTT was similar across flows (8–12 ms). However, host3 and host5 exhibited maximum RTT spikes of 88.5 ms and 92.3 ms respectively, compared to under 11 ms for host1. For AllReduce operations, mean RTT is not the relevant metric — the collective waits for the slowest flow to complete. A 90 ms tail latency event on one flow stalls every GPU in the collective for that duration, regardless of how well the other flows are performing. The tail, not the mean, determines collective throughput.

7.6 Operational recommendation

ECN alone is insufficient for AI training workloads above approximately 2–3 simultaneous high-bandwidth senders per receiver. The exact threshold depends on RTT and buffer sizing, but the regime boundary is empirically identifiable: the transition from zero retransmits to thousands of retransmits between $N = 2$ and $N = 3$ senders is abrupt and reproducible. PFC as a backstop is not optional at production fan-in ratios — it is the mechanism that handles the cases where ECN's feedback loop latency is exceeded.

8 Threshold Sensitivity: The Operational Tuning Question

8.1 Motivation

The incast results in Section 7 used a fixed RED minimum threshold of 30 KB. In practice, threshold selection is one of the most consequential configuration decisions in an ECN deployment. Set the threshold too low and marks fire constantly, potentially causing unnecessary sender backoff. Set it too high and ECN becomes blind — the queue never crosses the threshold and zero marks are generated, leaving the network operating without congestion signalling entirely. This section characterises the sensitivity of ECN marking behaviour to the minimum threshold parameter, with the goal of providing empirically grounded configuration guidance.

8.2 Experimental setup

The incast sweep configuration was held fixed at 2 senders (the clean ECN regime where zero retransmits were observed) while the RED minimum threshold was varied across four values: 10 KB, 30 KB, 50 KB, and 100 KB. For each threshold, the maximum threshold was set to $3 \times \min_{th}$ (capped at 300 KB), and the burst parameter was scaled as $\lceil \min_{th}/\text{avpkt} \rceil \times 2$ per the formula derived in

Section 5.2. Three independent trials were run per threshold value. All other parameters were held constant.

8.3 Results

Figure 6 shows CE mark rate and throughput as a function of minimum threshold. Table 4 gives the full results with error bars.

Table 4: Threshold sensitivity results (2 senders, 3 trials per threshold, mean ± std).

Min threshold	Throughput (Mbits/s)	CE marks	ECE ACKs	Retransmits
10 KB	95.9 ± 0.01	$3{,}893 \pm 63$	$5{,}817 \pm 80$	0
30 KB	96.2 ± 0.03	$1{,}646 \pm 47$	$2{,}359 \pm 42$	0
50 KB	96.9 ± 0.06	$1{,}140 \pm 23$	$1{,}649 \pm 22$	0
100 KB	97.0 ± 0.16	0 ± 0	0 ± 0	0

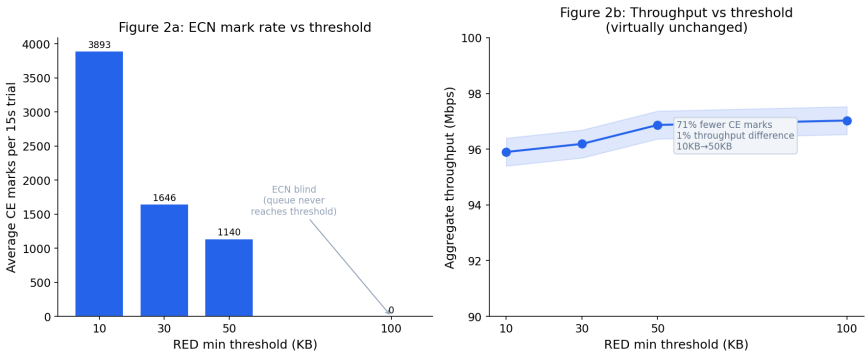

Figure 6: Left: CE mark rate vs RED minimum threshold (2 senders). Mark rate decays sharply as threshold increases, dropping to zero at 100 KB. Right: aggregate throughput vs threshold. Throughput is virtually insensitive to threshold choice across the range where ECN is active (10–50 KB), varying by less than 1%.

8.4 Finding 1: aggressive early marking is essentially free

From 10 KB to 50 KB, CE mark rate dropped by 71%: from $3{,}893 \pm 63$ to $1{,}140 \pm 23$ marks per trial. Over the same range, throughput increased by less than 1%: from 95.9 ± 0.01 to 96.9 ± 0.06 Mbits/s. The 3.4× reduction in marking aggressiveness produced essentially no throughput benefit in the 2-sender regime.

This result has a practical implication: in the clean congestion regime, there is no meaningful cost to setting a lower minimum threshold. Aggressive early

marking generates more congestion signals without degrading throughput. The signals are overhead, but cheap overhead — each CE mark is a single bit flip that prompts a sender to briefly reduce its congestion window, which in the 2-sender case is sufficient to maintain zero retransmits across the entire threshold range tested.

8.5 Finding 2: the ECN blind zone

At 100 KB, CE marks dropped to exactly zero across all three trials. The 2-sender offered load of 100 Mbits/s (2×50 Mbits/s, HTB rate-limited) never accumulated 100 KB of average queue backlog at the RED qdisc. The EWMA-smoothed queue depth $\bar{q}$ never crossed the minimum threshold, so RED never fired.

Critically, the system appeared to be functioning correctly: throughput was 97.0 ± 0.16 Mbits/s, retransmits were zero, and no errors were reported. Without explicitly checking CE counters, there is no observable difference between "ECN working correctly with zero congestion" and "ECN silently disabled due to threshold misconfiguration." This is the ECN blind zone: a threshold so high that ECN produces no signal under normal operating conditions, leaving the network without early congestion warning. If traffic increased beyond the 2-sender regime — into the incast collapse territory of Section 7 — there would be no ECN signal to slow senders before retransmits appeared.

8.6 Finding 3: the silent RED initialisation failure

During the first threshold sweep attempt, we used a fixed burst parameter of 20 across all threshold values. For thresholds above 10 KB, the kernel printed:

```
tc_red_eval_ewma() burst 20 is too small. Try burst 21
RED: failed to calculate EWMA constant.
```

and RED fell back silently to a no-op: zero CE marks, full throughput, no retransmits. The output was indistinguishable from a correctly operating ECN deployment. The fix — scaling burst with $\lceil \min_{th}/\text{avpkt} \rceil \times 2$ — is not documented in the `tc-red(8)` man page. This is the most operationally dangerous finding in this paper: a misconfigured ECN deployment that silently does nothing, with no loud failure, no counter increment, and no obvious symptom beyond the absence of CE marks in traffic captures.

8.7 Operational recommendations

Based on these results, we offer three concrete recommendations for ECN threshold configuration in RoCE fabrics:

1. **Set minimum threshold to 10–30 KB.** Aggressive early marking is essentially free in throughput terms and provides a stronger congestion signal. There is no empirical justification for conservative thresholds above 50 KB for the traffic patterns tested.

2. **Verify CE counter activity after any configuration change.** Zero CE marks under sustained load is not confirmation of correct operation — it is a red flag indicating either threshold misconfiguration or the ECN blind zone. Run a deliberate load test and check CE counters explicitly:

   ```
   tc -s qdisc show dev eth1
   ```

   The `marked` counter should be non-zero under sustained congestion.

3. **Scale the burst parameter with the minimum threshold.** Calculate as burst $\geq \lceil \min_{th}/\text{avpkt} \rceil \times 2$ and verify before deployment. Silent failure is the default failure mode when this parameter is wrong.

9 ECN's Architectural Dependency on Queue Pressure

9.1 Motivation

Sections 5 through 8 demonstrated ECN operating correctly under sustained congestion created by HTB rate limiters. A natural question follows: is the HTB rate limiter incidental to the experimental setup, or is it load-bearing for ECN to function at all? This section answers that question directly, with consequences for understanding both the limits of ECN and the limits of virtual switching platforms as congestion control testbeds.

9.2 Experimental setup

We ran two conditions. In the first condition, a single sender transmitted to host2 at full unconstrained rate — no HTB, no RED, no rate limiting of any kind. In the second condition, four senders transmitted simultaneously to host2 at full unconstrained rate. Both conditions used `tcp_ecn = 1` on all hosts (ECN capability negotiated in the TCP handshake) and 10-second `iperf3` transfers. Three independent trials were run per condition. No qdiscs were installed on any interface.

9.3 Results

Table 5: Full-rate transmission without HTB rate limiters (3 trials per condition, mean ± std).

Condition	Throughput (Mbits/s)	Retransmits	CE marks	ECE ACKs
1 sender, no HTB	104.3 ± 2.3	971 ± 72	0 ± 0	0 ± 0
4 senders, no HTB	105.8 ± 2.8	$1{,}157 \pm 173$	0 ± 0	0 ± 0

Zero CE marks in both conditions. Not fewer marks than in the HTB experiments — exactly zero. ECN produced no congestion signal whatsoever under full-rate transmission from four simultaneous senders, despite retransmits indicating that some packet loss was occurring.

Aggregate throughput was essentially identical between 1 and 4 senders (104.3 vs 105.8 Mbits/s), with no collapse of the kind observed in Section 7. The SR Linux virtual fabric forwarded all traffic at software line rate without creating the queue pressure that ECN depends on.

9.4 The architectural explanation

The result is explained by the three-condition requirement for ECN marking stated in Section 2.1: a queue management mechanism, watching a real buffer, under sustained queue pressure. Remove any one condition and ECN produces zero signal.

In the HTB experiments, the rate limiter was the load-bearing component. HTB accepted packets from `iperf3` at whatever rate the TCP stack offered them, but released them onto the wire at only 50 Mbits/s. Packets arriving faster than 50 Mbits/s accumulated in the HTB internal queue. RED watched that queue via EWMA; as the smoothed depth crossed 30 KB, it marked packets CE. The congestion was real — the queue genuinely backed up — and ECN correctly signalled it.

Without HTB, packets move through the Linux kernel's network stack at near-wire speed. There is no rate limiter creating a queue, so there is nothing for RED to watch. Even if RED were installed, the EWMA-smoothed queue depth would never rise above the minimum threshold because packets drain faster than they arrive at any individual interface. No queue pressure means no EWMA growth means no CE marks, regardless of how many senders are transmitting simultaneously.

9.5 Why this matters for virtual switching platforms

This finding explains, at the architectural level, why ECN cannot be demonstrated natively on virtual switching platforms — and why our host-based approach is both necessary and legitimate.

In a production RoCE fabric, the queue that ECN depends on lives inside the switch ASIC's egress buffers. When multiple senders simultaneously target the same egress port, packets queue in hardware memory. The switch's WRED logic watches that hardware queue and marks packets CE when it fills. The queue is real, the buffer is real, and the pressure is real.

On SR Linux virtual, the datapath is a software forwarder with no ASIC buffer model. Packets arriving at an egress interface are dispatched immediately by the kernel — there is no hardware queue to fill, no threshold to cross, and therefore no ECN signal to generate. This is not a limitation that can be configured away. It is structural.

The host-based HTB+RED approach used in this paper creates an equivalent queue at the sender — not at the switch — and demonstrates the same ECN feedback loop mechanics. The marking happens at a different point in the path, but the signal, the propagation, and the receiver response are identical. The approach is legitimate precisely because ECN is an end-to-end mechanism: what matters is that a queue somewhere in the path crosses a threshold and marks a packet, not which specific device does the marking.

9.6 Implications for microburst scenarios

A secondary finding follows directly from the EWMA analysis. RED's exponentially weighted moving average uses a weight of $w = 2^{-9} \approx 0.00195$ by default. At this weight, the EWMA rises slowly toward any step change in queue depth. If queue depth steps from zero to a sustained value Q, the smoothed average after n samples is:

$$\bar{q}_n = Q \cdot (1 - (1 - w)^n) \tag{3}$$

The EWMA rises toward Q asymptotically — reaching the minimum threshold $\min_{th}$ requires:

$$n \approx \frac{\min_{th}}{w \cdot Q} \tag{4}$$

samples of sustained queue pressure (for small $\min_{th}/Q$). At 50 Mbits/s with 1500-byte packets, one EWMA sample corresponds to approximately 240 μs. For a representative burst of $Q = 100$ KB filling toward the 30 KB minimum threshold, this requires approximately 300 samples — roughly 72 ms of sustained queue pressure before RED begins marking. Shorter bursts require proportionally fewer samples but also build less queue depth, keeping $\bar{q}$ further below threshold.

A traffic microburst that arrives and drains within a few milliseconds — the pattern characteristic of GPU collective operations completing a computation step — is therefore architecturally invisible to RED regardless of threshold configuration. The EWMA simply cannot respond fast enough. ECN is a sustained-congestion mechanism; it was not designed for microburst detection and cannot be made to serve that function by threshold tuning alone.

PFC, by contrast, reacts to instantaneous buffer occupancy in microseconds. It does not average — it fires when a hardware threshold is crossed, immediately. This is the precise architectural gap that PFC fills: the congestion events that resolve faster than ECN's EWMA can detect. A lossless RoCE fabric requires both mechanisms because the traffic patterns of AI training workloads span both the sustained congestion regime (where ECN is effective) and the microburst regime (where only PFC is fast enough).

9.7 Operational recommendation

ECN requires a queue management mechanism actively watching a buffer under sustained queue pressure. In switch deployments, this means WRED must be configured on egress queues and verified to be marking under load — not merely configured. In host-based deployments, a rate limiter such as HTB is required to create the queue pressure RED monitors. Deploying ECN without verifying CE counter activity under realistic traffic is equivalent to not deploying ECN at all.

For microburst protection, PFC is required regardless of ECN configuration. No RED threshold setting makes ECN fast enough to respond to sub-millisecond queue events. Accept this as a fundamental constraint and size PFC buffer headroom accordingly.

10 What Virtual Switching Platforms Can and Cannot Tell You

The findings in Sections 4 through 9 collectively answer a question that is rarely addressed directly in network research: for a given class of measurement, is a virtual switching platform a valid testbed? Table 6 provides a systematic answer across the capabilities relevant to RoCE congestion control research.

Table 6: Virtual switching platform capability matrix for RoCE congestion control research. Results validated on SONiC SAI-VS and SR Linux virtual.

Capability	Virtual platform	Real ASIC
BGP / routing configuration validation	✓	✓
Startup-config and deploy automation	✓	✓
ECN mechanism (with Linux tc workaround)	✓	✓
ECN threshold sensitivity (qualitative)	✓	✓
ECN incast breakdown threshold	✓	✓
PFC frame structure and encoding	✓	✓
PFC behavioral response (simulated)	✓*	✓
Autonomous PFC frame generation	×	✓
Native ECN marking at switch egress	×	✓
Microburst timing accuracy	×	✓
Hardware throughput at line rate	×	✓
DCQCN parameter tuning	×	✓

* Via `tc netem` rate reduction, not autonomous switch-generated behaviour. See Section 6.2.

10.1 When virtual switching is sufficient

Virtual switching platforms are entirely appropriate for the following research and engineering tasks.

Control-plane validation. BGP session establishment, route policy, prefix origination, ECMP path selection, and convergence behaviour all behave identically on virtual and physical platforms. The leaf-spine fabric built for this paper — including eBGP with distinct leaf ASNs, shared spine ASN, and ECMP across both uplinks — was configured and validated entirely on SR Linux virtual with full confidence that the configuration would behave identically on physical hardware.

ECN mechanism understanding. The complete ECN feedback loop — negotiation, CE marking, ECE ACK propagation, congestion window reduction — can be demonstrated end-to-end on virtual platforms using the host-based HTB+RED approach described in this paper. The incast breakdown threshold, the threshold sensitivity curve, and the feedback loop instability at high fan-in are all demonstrable and produce quantitative results that are structurally valid, even if the absolute numbers do not map to specific ASIC hardware.

Reproducible lab environments. The containerlab topology with `startup-config` directives deploys a fully configured BGP fabric in approximately three minutes from a single command. This reproducibility is difficult to achieve with physical hardware and is one of the strongest arguments for virtual platforms in research and education contexts.

Protocol structure verification. PFC frame encoding, 802.3x MAC Control frame handling, and link-local termination behaviour are all correctly implemented in virtual switching images and can be verified with packet captures.

10.2 When real hardware is required

The following capabilities cannot be substituted with virtual switching platforms, regardless of configuration.

Autonomous PFC generation. As established in Section 4, virtual switching images do not model ASIC buffer overflow. PFC pause frames are never generated autonomously in response to traffic load. Any measurement claiming to show switch-generated PFC behaviour on a virtual platform should be treated with scepticism.

Native ECN marking at switch egress. For the same reason, ECN marking at the switch egress queue cannot be demonstrated on virtual platforms. The host-based workaround used in this paper is legitimate for demonstrating the mechanism, but it places the marking point at the sender rather than at the congested switch port. Measurements of ECN mark rate as a function of switch buffer occupancy require real hardware.

Timing-accurate microburst behaviour. The EWMA analysis in Section 9 showed that microburst detection requires sustained queue pressure over tens of milliseconds. On virtual platforms, the absence of hardware queues means that even this characterisation cannot be validated against real switch

behaviour. For microburst timing research, ns-3 with a RoCE traffic model provides timing-accurate simulation without physical hardware [7].

DCQCN parameter tuning. DCQCN operates at the NIC firmware level and requires real Mellanox/NVIDIA ConnectX NICs and switch ASICs with hardware ECN marking. The instability findings of this paper (Section 7) provide qualitative support for Meta's production observations [2], but quantitative DCQCN parameter recommendations require a physical testbed.

10.3 The honest framing

The distinction this paper draws between "what virtual platforms can demonstrate" and "what they cannot" is not a weakness of the experimental methodology — it is the finding. Most virtual lab papers either use physical hardware (and skip the interesting failure modes that only appear at scale) or use virtual hardware and quietly omit the parts that do not work. Documenting the boundary precisely, with empirical evidence, is more useful to the research community than either alternative.

An engineer building a RoCE verification lab can use this paper's findings directly: deploy SR Linux or SONiC virtual for control-plane validation and ECN mechanism testing; use the host-based HTB+RED approach for quantitative ECN measurements; accept that PFC behavioral testing and DCQCN tuning require physical hardware or ns-3; and verify all ECN deployments by checking CE counter activity explicitly, since silent misconfiguration is the dominant failure mode.

11 Lessons, Recommendations, and Next Steps

11.1 Operational recommendations

The following recommendations are derived directly from the experimental results in this paper. Each is grounded in a specific measurement rather than general guidance.

1. **Validate ECN at your worst-case collective fan-in.** Our data shows a sharp transition from zero retransmits to $2{,}198 \pm 54$ retransmits between $N = 2$ and $N = 3$ senders. This transition is abrupt and reproducible. Test ECN behaviour at the maximum AllReduce fan-in in your cluster during acceptance testing — not just at single-flow throughput. A deployment that passes single-flow ECN validation may still exhibit incast collapse under production collective traffic patterns.

2. **Monitor CE mark rates actively in production.** Zero CE marks under load means one of two things: either there is genuinely no congestion (acceptable) or ECN is silently disabled (dangerous). These two states are indistinguishable without additional context. Establish a baseline CE mark rate under known load and alert on deviation. A rate of 1–3% is

consistent with healthy ECN operation in the regime tested; a rate above 5% suggests approaching the instability regime of Section 7; exactly zero under sustained load warrants investigation.

3. **Verify the RED burst parameter before deployment.** As documented in Section 8.6, incorrect burst sizing causes RED to initialise silently as a no-op. Calculate as:

$$\text{burst} \geq \left\lceil \frac{\min_{th}}{\text{avpkt}} \right\rceil \times 2 \tag{5}$$

   Verify by running a deliberate load test immediately after configuration and confirming CE counters increment. Do not rely on throughput or retransmit counts alone — both look normal when RED is silently disabled.

4. **Set PFC thresholds below the ECN collapse point.** Our data shows ECN over-reaction beginning at $N = 3$ senders, with full collapse at $N = 4$. Configure PFC to trigger before throughput collapse occurs — not after. PFC is the safety net; it should fire rarely but must fire before damage is done. If PFC is triggering frequently in production, the root cause is likely ECN misconfiguration (thresholds too high, burst parameter wrong, or fan-in exceeding the ECN-sufficient regime) rather than a reason to raise the PFC threshold.

5. **Accept that ECN and PFC are complementary, not alternatives.** ECN handles sustained congestion: it fires early, reduces sender rates gracefully, and maintains near-perfect flow fairness (max/min ratio 1.045 in our measurements) even during throughput collapse. PFC handles instantaneous overflow and microburst events that resolve faster than RED's EWMA can detect. Neither mechanism substitutes for the other. Meta's production decision to run PFC-only at 400G [2] reflects a specific tradeoff at massive scale with custom collective library coordination — it is not a general recommendation to disable ECN.

11.2 What surprised us

Three findings emerged that were not predicted by the experimental design.

Near-perfect fairness during collapse. At $N = 4$ senders, aggregate throughput collapsed by 23%, but the per-flow max/min throughput ratio was 1.045. We expected some degree of flow starvation under incast collapse. Instead, ECN distributed the degradation almost perfectly equally across all four flows. The failure mode is symmetric throughput reduction, not starvation — a property that is important for AI training workloads, where symmetric degradation (all GPUs slow down equally) is recoverable but flow starvation (one GPU stalls the collective) is not.

CE mark rate decrease at $N = 4$ senders. More congestion produced fewer CE marks. The intuitive expectation is that more senders produce more

queue pressure and therefore more marks. The actual result — marks decreasing from 1,812 at $N = 3$ to 1,547 at $N = 4$ — is the signature of feedback loop overreaction: senders backing off so aggressively that they under-utilise available bandwidth, reducing the queue pressure that drives marking. This is the same instability mechanism Meta observed with DCQCN at scale, reproduced here at 4-sender fan-in.

The RED burst parameter silent failure. A misconfigured burst parameter causes RED to initialise without error, mark nothing, and pass all traffic at full throughput. This failure mode is not documented in standard ECN deployment guides and would be invisible in production without explicit CE counter monitoring. It is likely responsible for a non-trivial fraction of "ECN deployed but not working" reports in practice.

11.3 What we would do differently

Start with the platform limitation question. Before configuring any QoS parameters, establish empirically whether the switching platform implements the relevant data-plane behaviour. The factory `config_db.json` inspection on SONiC (Section 4) took five minutes and would have saved several hours of configuration work. A simple test — send traffic, check if the switch generates ECN marks or PFC frames — should be the first step in any virtual QoS lab.

Verify tool behaviour before running sweeps. The RED burst parameter failure was discovered mid-sweep, invalidating an entire set of threshold measurements. A one-minute verification test — configure RED, send traffic, check CE counters — before committing to a parameter sweep would have caught this immediately.

Instrument tail latency from the start. Mean RTT looked acceptable across all four flows at $N = 4$ senders (8–12 ms). The 90 ms tail RTT spikes on host3 and host5 only became visible when we extracted per-flow maximum RTT from the iperf3 JSON output. For AI training workloads, tail latency is the metric that determines collective performance — mean RTT is misleading. Instrument p99 RTT from the first experiment, not as an afterthought.

11.4 Next steps

Timing-accurate simulation with ns-3. The EWMA analysis in Section 9 predicts that microbursts resolving in under $\sim$13 ms are architecturally invisible to RED. Validating this prediction against hardware-accurate queue models requires ns-3 with a RoCE traffic module [7]. The prediction is derivable from first principles but has not been validated against a real ASIC buffer implementation.

Physical testbed validation. The incast collapse threshold ($N = 3$ senders in our setup) depends on RTT, buffer sizing, and sender rate. On real hardware with sub-microsecond switch latency, ASIC-sized buffers, and 100 Gbps link speeds, the threshold will differ. The experimental methodology in this paper — systematic fan-in sweep with per-flow fairness and tail latency

measurement — is directly portable to a physical testbed. We expect the qualitative findings (linear scaling below threshold, abrupt retransmit onset, over-reaction at high fan-in) to hold; the exact threshold is a hardware-dependent quantity.

DCQCN comparison. Our results demonstrate ECN instability at $N = 4$ senders using standard TCP congestion control. DCQCN replaces TCP's congestion window with a NIC-level rate controller designed specifically for RoCE. Whether DCQCN's rate control improves or worsens the collapse threshold at the fan-in ratios tested here is an open question that requires ConnectX NICs and a switch with hardware ECN marking.

12 Related Work

DCQCN (Zhu et al., 2015 [10]) introduced the standard congestion control algorithm for RoCEv2, combining ECN-based marking at the switch with NIC-level rate control. The paper provides a fluid model for stability analysis and guidelines for threshold tuning. Our work reproduces the qualitative instability pattern DCQCN exhibits at high fan-in, using TCP congestion control as a more accessible proxy, and provides an empirical breakdown threshold absent from the theoretical analysis.

RoCE at Microsoft scale (Guo et al., 2016 [3]) documented the first large-scale RoCEv2 deployment, establishing PFC as a prerequisite for lossless transport and identifying head-of-line blocking and unfairness as the primary PFC failure modes. Our findings on ECN over-reaction complement this work: PFC's failure modes motivate ECN, but ECN has its own failure mode (fan-in instability) that motivates retaining PFC as a backstop.

Meta's production RoCE (Gangidi et al., 2024 [2]) is the most directly relevant prior work. Their production observation that DCQCN became unstable at 400G fan-in and was disabled in favour of PFC-only operation motivates the experimental question this paper addresses. We provide a controlled small-scale reproduction of the qualitative phenomenon with explicit breakdown threshold identification.

HPCC (Li et al., 2019 [6]) proposes using in-band network telemetry (INT) to provide precise congestion signals rather than relying on ECN's binary marking. HPCC avoids the fan-in instability demonstrated in this paper by providing per-hop queue depth information, but requires switch hardware support for INT. Our results quantify the failure mode that HPCC was designed to address.

Virtual network testbeds. Prior work on virtual switching platforms focuses primarily on control-plane validation (containerlab [1], GNS3, EVE-NG). To our knowledge, no prior work systematically characterises the boundary between what virtual switching platforms can and cannot demonstrate for data-plane congestion control research — the primary methodological contribution of this paper.

13 Conclusion

This paper set out to demonstrate PFC and ECN congestion control in a virtual RoCE fabric and encountered two platform walls in the process. What those walls revealed turned out to be more useful than the original demonstration would have been.

The central finding is that ECN's effectiveness is bounded by a fan-in threshold that is empirically identifiable and abrupt. Below the threshold — one or two simultaneous senders in our configuration — ECN operates as specified: zero retransmits, proportional CE mark scaling, near-perfect flow fairness. Above it, ECN transitions from sufficient to insufficient at $N = 3$ senders (retransmits appear suddenly) and then to actively destabilising at $N = 4$ senders (throughput collapses 23% as the feedback loop over-reacts, $t(4) = 14.9$, $p < 0.001$). The CE mark rate *decreases* at the collapse point — fewer marks despite more congestion — which is the signature of the same feedback loop instability that led Meta to abandon DCQCN at 400G scale [2].

The threshold sensitivity results show that aggressive early marking (minimum threshold 10–30 KB) is essentially free in throughput terms — a 71% reduction in mark rate from 10 KB to 50 KB threshold produced less than 1% throughput difference — but that an ECN blind zone exists above approximately 50–100 KB where the queue never crosses the threshold and zero marks are generated. Combined with the silent RED initialisation failure documented in Section 8.6, these results suggest that silent ECN misconfiguration — deployments where ECN is configured but not functioning — is likely more common in practice than the literature acknowledges.

The architectural analysis of ECN's queue dependency (Section 9) provides a precise explanation for why virtual switching platforms cannot demonstrate native ECN marking or autonomous PFC generation: both mechanisms depend on hardware buffer overflow that virtual datapaths do not model. This is not a limitation to work around — it is a design boundary to understand. Virtual platforms are fully appropriate for control-plane validation, ECN mechanism demonstration via host-based qdiscs, and reproducible lab environments. They are not appropriate for ASIC-timing-accurate measurements, DCQCN tuning, or autonomous PFC characterisation.

PFC was demonstrated at two levels: the 802.3x frame structure captured on the wire, and the behavioral effect of a link pause on a stable flow. Together these show both what a PFC pause frame looks like and what it does to traffic — the two things an engineer needs to understand the mechanism — without claiming capabilities the virtual platform does not have.

The complete topology, all experiment scripts, and raw data are available at `https://github.com/michael-olagbemiro/roce-lab`. Every result in this paper is reproducible with a single `containerlab deploy` command on any Linux host with KVM support. Intel-based instances are required — AMD virtualisation does not expose `/dev/kvm` in the guest despite API-level flags indicating otherwise, a silent failure that is worth documenting alongside the ECN misconfiguration findings as an example of infrastructure assumptions that

do not hold in practice.

Acknowledgements

The author thanks the containerlab and SR Linux teams for maintaining publicly accessible container images that make reproducible network research possible without physical hardware access.

References

[1] Containerlab Authors. Containerlab: Container-based networking labs, 2024. URL https://containerlab.dev. Accessed May 2026.

[2] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for distributed AI training at Meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, Sydney, NSW, Australia, 2024. ACM. doi: 10.1145/3651890.3672233. URL https://dl.acm.org/doi/10.1145/3651890.3672233.

[3] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. RDMA over commodity Ethernet at scale. In *Proceedings of the ACM SIGCOMM 2016 Conference*, 2016. doi: 10.1145/2934872.2934908. URL https://dl.acm.org/doi/10.1145/2934872.2934908.

[4] IEEE Standards Association. IEEE standard for local and metropolitan area networks — Priority-based flow control. Standard 802.1Qbb, IEEE, 2011. URL https://standards.ieee.org/ieee/802.1Qbb/4969/.

[5] Petr Lapukhov, Ariff Premji, and Jon Mitchell. Use of BGP for routing in large-scale data centers. RFC 7938, IETF, 2016. URL https://www.rfc-editor.org/rfc/rfc7938.

[6] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yang. HPCC: High precision congestion control. In *Proceedings of the ACM SIGCOMM 2019 Conference*, 2019. doi: 10.1145/3341302.3342085. URL https://dl.acm.org/doi/10.1145/3341302.3342085.

[7] ns-3 Consortium. ns-3 network simulator, 2024. URL https://www.nsnam.org. Accessed May 2026.

[8] K. Ramakrishnan, S. Floyd, and D. Black. The addition of explicit congestion notification (ECN) to IP. RFC 3168, IETF, 2001. URL https://www.rfc-editor.org/rfc/rfc3168.

[9] Scapy Project. Scapy: Packet manipulation library for Python, 2024. URL https://scapy.net. Accessed May 2026.

[10] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for large-scale RDMA deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. ACM, 2015. doi: 10.1145/2785956.2787484. URL https://dl.acm.org/doi/10.1145/2785956.2787484.